

Rapport Technique : Prédiction de Trafic Urbain par Apprentissage de Graphe Contextuel (GNN)

Projet NYC Traffic Prediction

5 janvier 2026

Résumé

Ce document présente l'état d'avancement et la feuille de route pour le développement d'un modèle de prédiction de trafic basé sur la structure urbaine. L'objectif est d'entraîner un *Graph Neural Network* (GNN) capable de prédire la congestion sur un segment routier en fonction de ses caractéristiques topologiques (OpenStreetMap) et contextuelles (Points d'Intérêts : parkings, événements, commerces), sans dépendre de séries temporelles historiques.

Table des matières

1	État des Lieux : Actifs et Données Disponibles	2
1.1	Structure du Graphe (Réseau Routier)	2
1.2	Données de Trafic (Vérité Terrain)	2
1.3	Outils de Traitement Existants	2
2	Approche Méthodologique : Graph Neural Networks (GNN)	2
2.1	Modélisation du Problème	2
3	Feuille de Route et Implémentation	3
3.1	Phase 1 : Enrichissement des Données (Feature Engineering)	3
3.2	Phase 2 : Construction du Dataset PyTorch Geometric	3
3.3	Phase 3 : Architecture du Modèle	3
3.4	Phase 4 : Entraînement et Validation	3
4	Conclusion et Prochaines Étapes	4

1 État des Lieux : Actifs et Données Disponibles

Nous disposons actuellement d'un jeu de données complet modélisant le réseau routier de New York et l'historique de trafic associé sur la période 2010-2013.

1.1 Structure du Graphe (Réseau Routier)

Le réseau est modélisé sous forme de graphe orienté $G = (V, E)$ statique provenant d'OpenStreetMap.

- **Nœuds** (V) : Définis dans `nodes.csv`. Chaque nœud représente une intersection avec un identifiant unique (`node_id`) et des coordonnées géographiques (`xcoord`, `ycoord`).
- **Arêtes** (E) : Définies dans `links.csv`. Chaque lien connecte deux nœuds et possède des attributs physiques :
 - `street_length` : Longueur du segment en mètres.
 - `osm_class` : Type de route (résidentielle, autoroute, etc.).
 - `osm_name` : Nom de la rue.
 - Géométrie : Angles de début et de fin (`begin_angle`, `end_angle`).

1.2 Données de Trafic (Vérité Terrain)

Les estimations de trafic sont dérivées de trajets de taxis et stockées dans des fichiers CSV volumineux (ex : `travel_times_2013.csv`).

- **Granularité** : Une estimation par heure pour chaque lien ayant enregistré au moins un passage.
- **Métriques** :
 - `travel_time` : Temps de parcours estimé (secondes).
 - `num_trips` : Nombre de taxis (indique la fiabilité de la mesure).

1.3 Outils de Traitement Existants

Un script de pré-traitement (`preprocess.py`) a été développé pour :

1. Charger le graphe en mémoire de manière vectorisée.
2. Calculer la vitesse moyenne pondérée ($V = \frac{\text{longueur}}{\text{temps}}$) pour chaque segment et chaque heure.
3. Classifier la congestion en 4 niveaux (Fluide, Normal, Dense, Congestionné).
4. Générer des fichiers JSON horaires pour la visualisation.

2 Approche Méthodologique : Graph Neural Networks (GNN)

L'hypothèse centrale du projet est que la congestion routière est une fonction de la **topologie du réseau** et du **contexte urbain environnant**. Contrairement aux approches classiques de séries temporelles (ARIMA, LSTM), nous utilisons un GNN pour apprendre la corrélation spatiale.

2.1 Modélisation du Problème

Soit un graphe où chaque arête e_{ij} (lien entre le nœud i et j) possède un vecteur de caractéristiques X_{ij} . Nous cherchons à apprendre une fonction f telle que :

$$Y_{ij}^{(t)} = f(G, X_{ij}, C_{ij}, T)$$

Où :

- $Y_{ij}^{(t)}$ est la vitesse ou la classe de congestion à l'instant t .

- G est la structure du graphe (matrice d'adjacence).
- X_{ij} sont les caractéristiques structurelles (type de route, largeur).
- C_{ij} sont les caractéristiques contextuelles (densité de POIs, proximité parking).
- T sont les variables temporelles (heure, jour).

3 Feuille de Route et Implémentation

Voici les étapes techniques nécessaires pour transformer les données actuelles en un pipeline d'entraînement GNN.

3.1 Phase 1 : Enrichissement des Données (Feature Engineering)

Les données actuelles manquent de contexte urbain. Il est impératif de croiser `links.csv` avec des données externes (via l'API Overpass ou la librairie Python `osmnx`).

Action : Créer un script `enrich_graph.py` pour générer, pour chaque `link_id`, les attributs suivants :

- **Densité Commerciale :** Nombre de restaurants/magasins dans un rayon de r mètres.
- **Infrastructure :** Distance au parking le plus proche.
- **Événementiel :** Proximité de salles de concert, stades ou théâtres.
- **Centralité :** Distance euclidienne par rapport au centre-ville.

3.2 Phase 2 : Construction du Dataset PyTorch Geometric

Conversion des données CSV et enrichies vers le format standard de `torch_geometric`.

Structure de l'objet Data :

- x (Nœuds) : Coordonnées géographiques normalisées.
- `edge_index` : Tenseur de connectivité $[2, N_{links}]$ dérivé de `begin_node_id` et `end_node_id`.
- `edge_attr` : Matrice des features enrichies (concaténation de `osm_class`, POIs, Time encoding).
- y (Target) : Vitesse moyenne (Régression) ou Classe de congestion (Classification).

3.3 Phase 3 : Architecture du Modèle

Nous implémenterons un GNN capable de traiter les attributs des arêtes.

Architecture proposée (GraphSAGE ou GAT) :

1. **Input Layer :** Encodage des attributs des arêtes.
2. **Message Passing Layers** (x2 ou x3) : Agrégation de l'information des segments voisins. Cela permet au modèle de comprendre que "si les routes voisines qui mènent au parking sont bouchées, je risque de l'être aussi".
3. **Readout Layer :** Couche linéaire finale pour prédire la vitesse sur le lien spécifique.

3.4 Phase 4 : Entraînement et Validation

- **Loss Function :** MSE (Mean Squared Error) pour la vitesse exacte.
- **Stratégie de Validation Spatiale :** Pour prouver la robustesse du modèle, le jeu de test ne sera pas une période future, mais **une zone géographique distincte** (ex : Entraînement sur Brooklyn, Test sur Queens). Cela validera la capacité du modèle à généraliser l'impact de l'urbanisme sur le trafic.

4 Conclusion et Prochaines Étapes

Le projet dispose d'une base solide (graphe propre et données de trafic). La valeur ajoutée résidera dans la Phase 1 (Enrichissement contextuel).

Prochaine action immédiate : Écrire le script d'extraction des POIs (Points d'Intérêts) autour des coordonnées des nœuds existants pour enrichir le vecteur de features.